

TP1 - Fundamentos de IA

Gilberto Pereira
Jaime Guimaraes
Luigi Jacob
Yaggo Teixeira

September 2026

1 Tarefa 1 – Modelagem do Problema

O problema da Ponte e da Tocha pode ser modelado como um problema de busca em um grafo de estados. Seja o grafo ponderado

$$G = (V, E),$$

em que V representa o conjunto de estados possíveis do problema e E representa o conjunto de transições válidas entre esses estados. Cada aresta possui um custo associado, correspondente ao tempo necessário para realizar a travessia que produz a transição.

Dessa forma, a solução do problema consiste em encontrar um caminho no grafo que conecte o estado inicial ao estado objetivo. Como as travessias possuem tempos diferentes, os custos associados às arestas também podem ser diferentes.

1.1 Representação de Estado

Cada estado deve armazenar a posição das quatro pessoas e da tocha. Para isso, um estado S é representado pela tupla

$$S = (A, B, C, D, T),$$

em que A , B , C e D representam as quatro pessoas e T representa a posição da tocha.

Cada elemento da tupla assume um valor binário:

- 0: margem inicial;
- 1: margem final.

Portanto, o conjunto de estados possíveis pode ser definido como

$$V \subseteq \{0, 1\}^5.$$

Por exemplo, o estado

$$S = (1, 1, 0, 0, 1)$$

indica que as pessoas A e B , juntamente com a tocha, encontram-se na margem final da ponte, enquanto C e D permanecem na margem inicial.

É importante observar que os valores binários representam a **posição atual** de cada elemento, e não simplesmente se uma pessoa já realizou uma travessia. Isso é necessário porque, durante a solução do problema, pessoas podem atravessar a ponte em ambos os sentidos.

1.2 Estado Inicial e Estado Objetivo

No início do problema, todas as pessoas e a tocha encontram-se na margem inicial. Assim, o estado inicial é

$$S_0 = (0, 0, 0, 0, 0).$$

O objetivo consiste em levar todas as quatro pessoas para a margem final. Como toda travessia necessita da tocha, ao término da solução ela também estará nessa margem. Portanto, o estado objetivo é

$$S_G = (1, 1, 1, 1, 1).$$

No grafo de estados, S_0 corresponde ao vértice de origem da busca e S_G ao vértice objetivo.

1.3 Função Sucessora

A função sucessora determina, para um determinado estado S , quais estados podem ser alcançados por meio de uma única travessia válida.

Seja

$$S = (A, B, C, D, T).$$

Primeiramente, identifica-se o conjunto P_T formado pelas pessoas que estão na mesma margem que a tocha:

$$P_T = \{p \in \{A, B, C, D\} \mid p = T\}.$$

Uma ação válida consiste em selecionar uma ou duas pessoas pertencentes a P_T . Assim, para um conjunto de pessoas M selecionado para realizar a travessia, deve-se satisfazer

$$M \subseteq P_T$$

e

$$1 \leq |M| \leq 2.$$

A função sucessora gera todas as combinações válidas de uma ou duas pessoas que estejam na mesma margem que a tocha. Cada uma dessas combinações produz um novo estado e, conseqüentemente, uma aresta no grafo.

Formalmente, pode-se representar a função sucessora como

$$Succ(S) = \{(M, S', c(M)) \mid M \subseteq P_T, 1 \leq |M| \leq 2\},$$

em que M representa a ação realizada, S' representa o estado resultante e $c(M)$ representa o custo da travessia.

1.4 Ações e Transições

Uma ação consiste no deslocamento de uma ou duas pessoas da margem em que se encontra a tocha para a margem oposta.

Quando uma pessoa participa da travessia, sua posição é invertida. Se x_p representa a posição da pessoa p , então

$$x'_p = 1 - x_p, \quad \forall p \in M.$$

As pessoas que não participam da travessia permanecem na mesma margem:

$$x'_p = x_p, \quad \forall p \notin M.$$

Como toda travessia exige a presença da tocha, sua posição também é invertida:

$$T' = 1 - T.$$

Dessa forma, cada ação válida define uma transição entre dois vértices do grafo.

Por exemplo, considerando o estado inicial

$$(0, 0, 0, 0, 0),$$

caso as pessoas A e B atravessem juntas, ocorre a transição

$$(0, 0, 0, 0, 0) \xrightarrow{\{A, B\}} (1, 1, 0, 0, 1).$$

O novo estado constitui outro vértice do grafo e a travessia realizada corresponde à aresta que conecta os dois estados.

1.5 Custos das Transições

Cada pessoa possui um tempo específico de travessia:

$$t(A) = 1, \quad t(B) = 2, \quad t(C) = 5, \quad t(D) = 10.$$

Quando apenas uma pessoa atravessa, o custo da transição corresponde ao seu próprio tempo:

$$c(\{p\}) = t(p).$$

Quando duas pessoas atravessam juntas, ambas precisam se deslocar na velocidade da pessoa mais lenta. Portanto, o custo da transição é dado por

$$c(M) = \max_{p \in M} t(p).$$

Por exemplo,

$$c(\{A, B\}) = \max(1, 2) = 2,$$

enquanto

$$c(\{A, D\}) = \max(1, 10) = 10.$$

Assim, cada transição do grafo pode ser representada por uma aresta ponderada

$$S \xrightarrow[M]{c(M)} S',$$

na qual S é o estado de origem, S' é o estado resultante, M é a ação realizada e $c(M)$ é o peso da aresta.

Consequentemente, uma solução para o problema corresponde a um caminho

$$S_0 \rightarrow S_1 \rightarrow \cdots \rightarrow S_G$$

no grafo de estados. O custo total desse caminho é a soma dos custos das travessias realizadas:

$$C = \sum_{i=0}^{k-1} c(S_i, S_{i+1}).$$

O objetivo final consiste, portanto, em encontrar um caminho entre S_0 e S_G , sendo que, para os métodos de busca orientados por custo, busca-se especificamente o caminho cujo custo total seja mínimo. ““

2 Algoritmos de Busca

Para solucionar o problema da Ponte e da Tocha, foram considerados quatro algoritmos de busca: Busca em Profundidade (DFS), Busca em Largura (BFS), Busca de Custo Uniforme (UCS) e Busca A*.

Os algoritmos utilizam o mesmo grafo de estados, diferenciando-se pelo critério adotado para selecionar o próximo nó a ser explorado.

2.1 Busca em Profundidade (DFS)

A Busca em Profundidade, ou Depth-First Search (DFS), é um algoritmo de busca não informada que explora um ramo do grafo até encontrar o estado objetivo ou até não ser possível avançar para novos nós.

Quando não existem sucessores ainda não explorados, o algoritmo retrocede para investigar outros caminhos, realizando o processo conhecido como back-tracking.

O DFS utiliza uma estrutura de pilha LIFO (Last In, First Out), na qual o último elemento inserido é o primeiro a ser removido.

Como não considera os custos das transições para determinar a ordem de exploração, o DFS não garante encontrar a solução de menor custo.

Além disso, em grafos que possuem ciclos, é necessário controlar os estados visitados para evitar que o algoritmo explore repetidamente os mesmos caminhos.

2.2 Busca em Largura (BFS)

A Busca em Largura, ou Breadth-First Search (BFS), é um algoritmo de busca não informada que explora os nós do grafo em ordem crescente de profundidade.

Diferentemente do DFS, o BFS explora todos os nós de um determinado nível antes de avançar para o nível seguinte.

Para isso, utiliza uma estrutura de fila FIFO (First In, First Out), na qual o primeiro elemento inserido é o primeiro a ser removido.

Em um grafo finito, com controle adequado dos estados visitados, o BFS encontra uma solução, caso exista um caminho entre o estado inicial e o estado objetivo.

O BFS garante encontrar uma solução com o menor número de transições. Entretanto, como os custos das travessias podem ser diferentes, essa solução não necessariamente apresenta o menor tempo total.

2.3 Busca de Custo Uniforme (UCS)

A Busca de Custo Uniforme, ou Uniform-Cost Search (UCS), é um algoritmo de busca não informada que considera os custos das transições para determinar a ordem de exploração dos nós.

Diferentemente do BFS, que prioriza os nós de menor profundidade, o UCS seleciona o nó com o menor custo acumulado desde o estado inicial.

Esse custo é representado pela função

$$g(n) = \sum_{i=0}^{k-1} c(S_i, S_{i+1}),$$

em que $g(n)$ corresponde ao custo acumulado para alcançar o nó n e $c(S_i, S_{i+1})$ representa o custo da transição entre dois estados consecutivos.

O algoritmo utiliza uma fila de prioridade, na qual os nós são ordenados pelo valor de $g(n)$.

Como os custos das travessias são positivos, o UCS garante encontrar uma solução de custo mínimo quando implementado corretamente.

No problema da Ponte e da Tocha, isso significa encontrar uma sequência de travessias que minimize o tempo total necessário para transportar todas as pessoas até a margem final.

2.4 Busca A*

A Busca A* é um algoritmo de busca informada que combina o custo acumulado desde o estado inicial com uma estimativa do custo restante para alcançar o estado objetivo.

A seleção dos nós é realizada por meio da função

$$f(n) = g(n) + h(n),$$

em que $g(n)$ representa o custo acumulado para alcançar o nó atual e $h(n)$ representa uma estimativa do custo necessário para atingir o estado objetivo.

O algoritmo utiliza uma fila de prioridade para selecionar o nó com menor valor de $f(n)$.

Dessa forma, o A* considera tanto o custo do caminho já percorrido quanto uma estimativa do custo restante, buscando orientar a exploração para caminhos com menor custo total estimado.

A função heurística $h(n)$ deve ser admissível, ou seja, não pode superestimar o custo mínimo real necessário para alcançar o objetivo.

Para solucionar o problema da Ponte e da Tocha, propõe-se uma função heurística baseada nos tempos de travessia dos indivíduos e na posição atual da tocha. Seja O o conjunto de pessoas na margem de origem e D o conjunto de pessoas na margem de destino. A função heurística $h(n)$ é definida da seguinte forma:

- Se a tocha estiver na **margem de origem**:

$$h(n) = \max_{p \in O} t(p)$$

- Se a tocha estiver na **margem de destino** (e $O \neq \emptyset$):

$$h(n) = \max_{p \in O} t(p) + \min_{p \in D} t(p)$$

Caso o estado seja o objetivo ($O = \emptyset$), $h(n) = 0$.

Essa heurística é **admissível**. No primeiro caso, para que o objetivo seja alcançado, todas as pessoas no conjunto O precisam obrigatoriamente atravessar a ponte. Como a ponte suporta no máximo duas pessoas e a travessia ocorre na velocidade da pessoa mais lenta, o custo mínimo para esvaziar a origem será sempre igual ou superior ao tempo da pessoa mais lenta que lá está.

No segundo caso, se a tocha encontra-se no destino e ainda existem pessoas na origem, tem-se a garantia de que pelo menos uma viagem de retorno precisará ocorrer. O retorno mais rápido possível à margem de origem é realizado pela pessoa mais rápida presente no destino ($\min_{p \in D} t(p)$). Somando-se essa viagem otimista com o limite inferior de travessia dos indivíduos restantes na origem ($\max_{p \in O} t(p)$), obtém-se um valor que nunca superestima o custo real restante.

Com uma heurística admissível, custos positivos e tratamento adequado de caminhos alternativos, o A* garante encontrar uma solução de custo mínimo.

Quando a função heurística assume valor zero para todos os estados, o A* utiliza apenas o custo acumulado e passa a apresentar o mesmo critério de expansão da Busca de Custo Uniforme.

3 Tarefa 4 – Comparação dos Métodos de Busca

Para comparar os quatro métodos implementados (DFS, BFS, UCS e A*), o programa foi executado sobre o mesmo grafo de estados e as seguintes métricas foram registradas: o custo da solução encontrada (tempo total de travessia), o número de nós expandidos durante a busca e o tempo de processamento até a obtenção da solução. Além disso, todas as buscas possuem um limite fixo de nós expandidos (no nosso caso, 10 000 nós), após o qual o programa é encerrado. Isso evita buscas excessivamente longas ou mesmo laços infinitos, o que é especialmente importante no DFS, que pode mergulhar em ramos muito profundos antes de encontrar uma solução.

Para o tempo de processamento, cada algoritmo foi executado 1 000 vezes e registraram-se a média e o desvio padrão das medições, feitas com a função `time.perf_counter()` da biblioteca padrão do Python. Os resultados obtidos estão resumidos na Tabela 1 e ilustrados nas Figuras 1 e 2

Algoritmo	Custo (min)	Nós expandidos	Tempo médio (μ s)
DFS	90	13	9,13 $\pm$ 0,85
BFS	19	26	13,19 $\pm$ 0,59
UCS	17	26	18,53 $\pm$ 2,63
A*	17	17	25,82 $\pm$ 3,61

Table 1: Comparação dos quatro métodos de busca (média de 1 000 execuções).

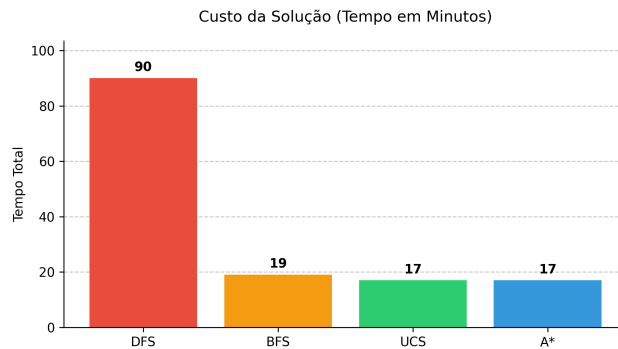


Figure 1: Comparação do custo total da solução (tempo em minutos) para cada algoritmo.

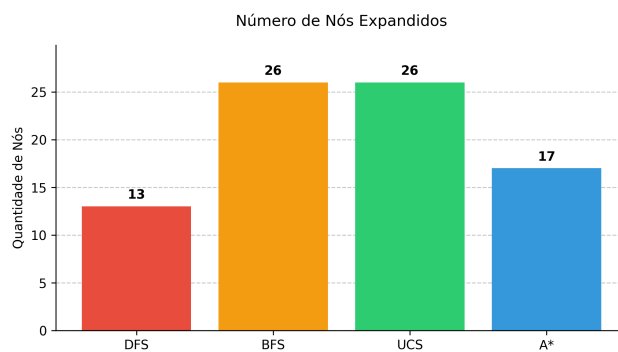


Figure 2: Comparação da quantidade de nós explorados por cada algoritmo.

3.1 Análise dos Resultados

Custo da solução. O DFS encontrou a pior solução (90 minutos): por não considerar nem a profundidade nem o custo das arestas, ele seguiu o primeiro ramo explorado em profundidade e retornou a primeira solução encontrada, que continha 10 travessias, muitas delas de alto custo. O BFS encontrou uma solução de 19 minutos, com o menor número de transições (5 travessias), mas ainda subótima em tempo. O UCS e o A* encontraram a solução ótima de 17 minutos.

Nós expandidos. O DFS expandiu apenas 13 nós, mas essa eficiência é acidental: ele teve a “sorte” de tropeçar em uma solução cedo, sem qualquer garantia sobre sua qualidade. BFS e UCS expandiram 26 nós cada, explorando o espaço de estados de forma sistemática. O A* expandiu apenas 17 nós, cerca de 35% a menos que o UCS, pois a heurística admissível orientou a busca em

direção ao objetivo, podando caminhos que, embora parcialmente baratos, não levariam à solução ótima.

Tempo de processamento. Como o grafo do problema possui apenas 32 estados, o tempo de processamento é da ordem de microssegundos para todos os algoritmos. Curiosamente, o A* apresentou o maior tempo médio ($25,82 \mu s$), apesar de expandir menos nós. Isso ocorre porque, neste problema pequeno, o custo das operações da fila de prioridade (heap) e do cálculo da função heurística a cada expansão supera o ganho obtido pela redução do número de nós explorados. De forma análoga, o DFS, que usa apenas uma pilha simples, foi o mais rápido ($9,13 \mu s$), ainda que tenha retornado a pior solução. Em problemas com espaços de estados maiores, espera-se que o número de nós expandidos seja o fator dominante do tempo total, e a redução obtida pelo A* (de 26 para 17 nós, cerca de 35% a menos que o UCS) tende a compensar esse custo adicional por nó.

3.2 Por que o BFS não garante a solução ótima?

O BFS garante encontrar a solução com o **menor número de transições** (arestas), pois explora os nós em ordem crescente de profundidade. Entretanto, essa garantia só coincide com a solução de menor custo quando todas as arestas têm o mesmo custo. No problema da Ponte e da Tocha, as travessias têm custos diferentes ($t(A) = 1$, $t(B) = 2$, $t(C) = 5$, $t(D) = 10$), de modo que um caminho com menos passos pode ser mais caro que um caminho com mais passos. De fato, o BFS retornou uma solução de 19 minutos com 5 travessias, enquanto a solução ótima (17 minutos) também utiliza 5 travessias, mas com escolhas de pares que minimizam o tempo total – o BFS, cego aos custos, não tem como fazer essa distinção: ele retorna o primeiro estado objetivo que atinge na profundidade mínima, qualquer que seja seu custo acumulado.

Já o UCS expande sempre o nó de menor custo acumulado $g(n)$. Como todos os custos das travessias são positivos, quando o estado objetivo é removido da fila de prioridade pela primeira vez, não existe nenhum caminho ainda não explorado capaz de alcançá-lo com custo menor – o que garante a otimalidade. O A* mantém essa garantia ao expandir o nó de menor $f(n) = g(n) + h(n)$, desde que a heurística $h(n)$ seja admissível (nunca superestime o custo real restante), o que é o caso da heurística utilizada, conforme demonstrado na seção anterior. A vantagem do A* sobre o UCS é que a heurística direciona a expansão, reduzindo o número de nós explorados (17 contra 26) sem abrir mão da solução ótima.

3.3 Conclusão

Os resultados confirmam a teoria: DFS e BFS são métodos não informados que não garantem a solução de menor custo em grafos com arestas de custos distintos, enquanto UCS e A* garantem a otimalidade. O A* destaca-se como o melhor método para este problema, pois encontra a solução ótima (17 minutos) expandindo o menor número de nós, graças à heurística admissível utilizada. Embora seu tempo médio de processamento tenha sido ligeiramente superior ao

do UCS neste grafo pequeno – devido ao custo da fila de prioridade e do cálculo da heurística –, a economia de nós expandidos tende a se tornar a vantagem decisiva à medida que o espaço de estados cresce. O limite fixo de nós expandidos atuou como salvaguarda para impedir buscas excessivamente longas, especialmente no DFS, que em grafos maiores ou com ciclos poderia se perder em ramos muito profundos sem encontrar solução.